SCOMP ADC Peripheral Project Summary

Kien Le

Submitted
April 28, 2026

Introduction

This document describes the ADC peripheral created for the SCOMP system, enabling programmers to incorporate potentiometers into the environment. The peripheral provides a easy-to-use interface that simplifies potentiometer and ADC communication for the user, including error indicators (0xF000) for invalid configuration or unresponsive channels.

Our approach prioritizes accessing the 8 different channels and implementing the read/write functionality via memory-mapped writes to addresses 0xC0-0xC7. While able to develop the peripheral's core features, it was ultimately unsuccessful due to communication mismatches between the peripheral and the SCOMP system. Time constraints and conflicting schedules limited our ability to debug and resolve these integration issues before the demonstration.

Device Functionality

The peripheral device takes 8 I/O addresses (`0xc0-0xc7`) corresponding to the 8 analog channels that the ADC can generate data from.

The peripheral creates the illusion of all 8 channels being available at the same time, but the ADC can only work on one channel at a time. This abstraction scheme introduces complications that we'll explain in subsections below.

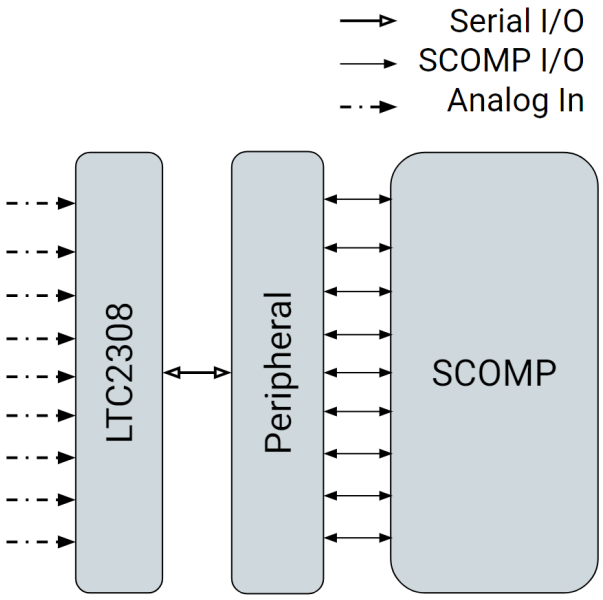

Figure 1. A diagram illustrating the peripheral's 8-channel abstraction scheme.

Reading from the Device (`IN`)

To read from a channel, execute `IN` on the I/O address for that channel. This will fill the accumulator with the 12-bit converted data last polled from that channel.

As the SCOMP is 16-bit, the upper 4 bits will be set to 0 on a successful read. However, in a short period of time after peripheral reset, data on some channels will be invalid because they haven't yet been polled. When reading from a channel with invalid data, the upper 4 bits of the accumulator will be set to 1 to indicate an error.

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Field	ERR_CODE				DATA											
Valid Data Example	0	0	0	0	X											
Invalid Data Example	1	1	1	1	X											

Table 1. I/O data breakdown demonstrating that the highest 4 bits is used as an error indicator.

Due to the latency introduced by the peripheral's abstraction scheme, valid data does not always reflect the voltage that is currently on that analog input. For the best performance, the SCOMP software should wait at least 1 second after the input voltage is changed.

Writing to the Device (`OUT`)

The device implements the ADC's unipolar/bipolar mode configuration option. Executing `OUT` to any of the 8 I/O addresses with a non-zero value in the accumulator will change the polarity mode of future conversions. A positive value sets unipolar mode, and vice versa.

Due to the device's abstraction scheme, mode changes may not be immediately reflected in the data.

Design Decisions and Implementation

We chose to use 8 I/O addresses because this design presents data along with its channel without using additional I/O resources. However, this meant we couldn't support the LTC2308's differential mode, which uses 2 channels. We therefore hard-coded the peripheral to always configure single-ended mode on the ADC.

We also chose to abstract away the notion of active channels. This means the user doesn't need to manually activate a channel or keep track of which channel is currently active. This led us to implement a round-robin polling system that configures the ADC to retrieve data from each channel cyclically.

Significant Modifications Made During Design

The original design was an active channel selection interface. The user would have activated a channel by executing an `OUT` to that channel's I/O address, and the peripheral would have only polled data from that channel.

While this design might have reduced latency, we ultimately didn't move forward with it because of the complexity it adds to the project and the time constraint we had.

Conclusions

While the peripheral implementation failed to execute the demonstration program, valuable lessons were gained from the planning, design, and prototype development phases. Developing the peripheral involved learning how the SPI interface worked in the communication pathway from analog to digital data. One key recommendation would be to include embedded timing detection to detect system hangs when a timeout

occurs with the ADC.

Additionally, for future applications of this peripheral, engineers integrating this peripheral should implement software debouncing for potentiometer noise. In retrospect, instead of the polling system, incorporating the active channel would better suit applications using only 1-2 channels rather than parsing through all eight continuously.